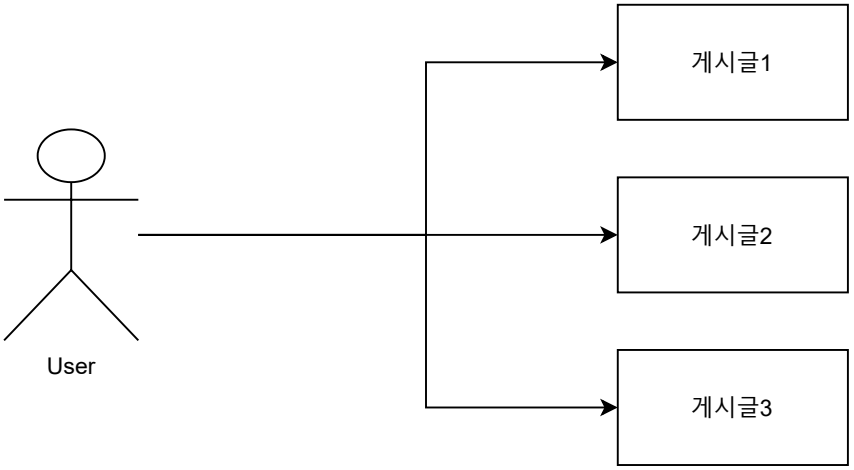
유저와 게시물의 관계 형성 해주기

현재는 유저나 게시물에 생성할 때 그 둘의 관계에 대해서는 설정해준게 없습니다. 게시물을 생성 할 때도 어떤 유저가 생성해줬는지 정보를 같이 넣어주어야 합니다. 그래서 이 유저와 게시물 관계 부분을 처리해주겠습니다.

유저와 게시물 데이터의 관계 형성

1. 관계를 형성하기 위해서는 엔티티에 서로간의 필드를 넣어줘야 합니다.



OneToMany Relationship

ManyToOne Relationship

https://typeorm.io/#/many-to-one-one-to-many-relations

A one-to-many relation allows creating the type of relation where Entity1 can have multiple instances of Entity2, but Entity2 has only one Entity1. Entity2 is the owner of the relationship, and stores the id of Entity1 on its side of the relation.

A many-to-one relation allows creating the type of relation where Entity1 can have a single instance of Entity2, but Entity2 can have multiple instances of Entity1. Entity1 is the owner of the relationship, and stores the id of Entity2 on its side of the relation.

```typescript
export class User extends BaseEntity {
    @PrimaryGeneratedColumn()
    id: number;

    @Column()
    username: string;

    @Column()
    password: string;

    @OneToMany(type => Board, board => board.user, { eager: true })
    boards: Board[];
```

```typescript
export class Board extends BaseEntity {
    @PrimaryGeneratedColumn()
    id:number;

    @Column()
    title: string;

    @Column()
    description: string;

    @Column()
    status: BoardStatus;
```

```
async validatePassword(password: string): Promise<boolean> {
    let isValid = await bcrypt.compare(password, this.password)
    return isValid;
}
```

```
    @ManyToOne(type => User, user => user.boards, { eager: false })
    user: User;
}
```



파라미터

1. Type

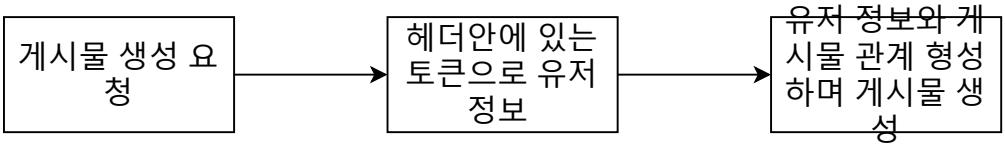
2. inverseSide (board에서 유저로 접근하려면 board.user로 접근해야함)

3. Option (eager : true 일때는 user 정보를 가져올때 board도 같이 가져옴)

```
typeFunctionOrTarget: string | ((type?: any) => ObjectType<User>),
inverseSide?: string | ((object: User) => any),
options?: RelationOptions
```

게시물 생성 할 때 유저 정보 넣어주기

유저와 게시물의 관계를 엔티티를 이용해서 형성해주었습니다.
그래서 이제는 실제로 게시물을 생성할 때 유저 정보를 게시물에 넣어주겠습니다




```
@Post()
@UsePipes(ValidationPipe)
createBoard(
  @Body() createBoardDto: CreateBoardDto,
  @GetUser() user:User,
): Promise<Board> {
  return this.boardsService.createBoard(createBoardDto, user);
}
```

```
async createBoard(
  createBoardDto: CreateBoardDto,
  user: User,
  ): Promise<Board> {
  return this.boardRepository.createBoard(createBoardDto, user);
}
```

```
export class BoardRepository extends Repository<Board> {
    async createBoard(
        createBoardDto: CreateBoardDto,
        user: User,
        ): Promise<Board> {
        const { title, description } = createBoardDto;
```

```
    const board = new Board();
    board.title = title;
    board.description = description;
    board.status = BoardStatus.PUBLIC;
    board.user = user;
    await board.save();

    delete board.user;

    return board;
  }
```

해당 유저의 게시물만 가져오기(getAllBoards)

현재는 게시물을 가져올 때 어떠한 유저인지에 상관 없이 모든 게시물을 가져옵니다. 그 부분을 게시물을 가져올 때 해당 유저가 생성한 게시물만 가져오도록 해주겠습니다.

```
@Get()
getAllBoards(
  @GetUser() user: User,
): Promise<Board[]> {
  return this.boardsService.getAllBoards(user);
}
```

```
async getAllBoards(
  user: User,
):  Promise <Board[]> {
  const query = this.boardRepository.createQueryBuilder('board');

  query.where('board.userId = :userId', { userId: user.id} )

  const boards = await query.getMany();
  return boards;
}
```

해당 유저의 게시물을 ID로 가져오기

게시물을 ID를 이용해서 가져올 때 해당 유저의 게시물만 가져올 수 있게 해주겠습니다.

```
@Get('/:id')
getBoardById(
  @Param('id', ParseIntPipe) id: number,
  @GetUser() user:User,
): Promise<Board> {
  return this.boardsService.getBoardById(id), user;
}
```

```
async getBoardById(
  id: number,
  user: User
): Promise<Board> {
  const found = await this.boardRepository.findOne({ where: { id, userId: user.id } });

  if (!found) {
    throw new NotFoundException(`Can't find Board with id ${id}`)
  }

  return found;
}
```

```
EntityColumnNotFound: No entity column "userId" was found.
    at new EntityColumnNotFound (/Users/jaewon/Documents/n
r/EntityColumnNotFound.ts:8:9)
```

```
export class Board extends BaseEntity {
    @PrimaryGeneratedColumn()
    id:number;

    @Column()
    title: string;
```

```
    @Column()
    description: string;

    @Column()
    status: BoardStatus;

    @ManyToOne(type => User, user => user.
    user: User;

    @Column()
    userId: number;

}
```

404 Error VS 403 Error

404 에러는 Not Found
403 에러는 forbidden

자신이 생성치 않은 게시물에 접근시 원래는 403 Error를 받아야 할 것 같은데 404 에러를 보내는 이유는? 403라는 에러 자체도 정보가 되기에 이 정보 또한 노출시키지 않기 위해서 404에러를

Pretty Raw Preview Visualize JSON

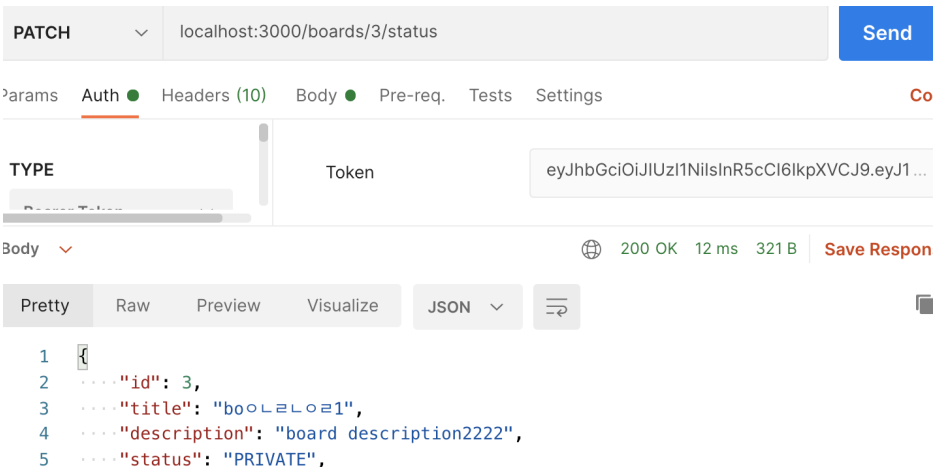

```
{
    "statusCode": 404,
    "message": "Can't find Board with id 1",
    "error": "Not Found"
}
```

자신이 생성한 게시물을 수정하기

수정하고자 원하는 게시물의 아이디를 이용해서 자신이 작성한 게시물만 수정을 해주겠습니다.

```
@Patch('/:id/status')
updateBoardStatus(
  @Param('id', ParseIntPipe) id: number,
  @Body('status', BoardStatusValidationPipe) status: BoardStatus,
  @GetUser() user: User
): Promise<Board> {
  return this.boardsService.updateBoardStatus(id, status, user);
}
```

```
async updateBoardStatus(
  id: number,
  status: BoardStatus,
  user: User
): Promise<Board> {
  const board = await this.getBoardById(id, user);
  board.status = status;
  await board.save();
  return board;
}
```



```
    "userId": 2
}
```

자신이 생성한 게시물을 삭제하기

자신이 생성한 게시물만을 삭제 할 수있게 기능을 구현해보겠습니다.

```
@Delete('/:id')
deleteBoarX$d(
  @Param('id', ParseIntPipe) id: number,
  @GetUser() user: User
  ): Promise<void> {
  return this.boardsService.deleteBoard(id, user);
}
```

```
async deleteBoard(id: number, user: User): Promise<void> {
  const result = await this.boardRepository.delete({id , userId: user.id});

  if (result.affected === 0) {
    throw new NotFoundException(`Can't find Board with id ${id}`)
  }
}
```